



FIT2096 - Games Programming

WEEK 02 LAB

Working with Actors

Welcome to Week 2 of Games Programming! In this lab, we're going to learn how to deal with actors and actor components within Unreal Engine, and ways to use the different parent classes available in Unreal.

We will create an actor component to manage our inventory system from Week 1, set up a custom input using Unreal's Enhanced Inputs, and create some new actors for our player to interact with, including power ups and an environmental hazard.

This lab is going to build off of the Week 01 Lab, so you should ensure that work is completed before moving on to this lab.

This Lab is Assessed

Each of the labs from weeks 1 - 9 are assessed as part of the Learning Portfolio. Please make sure to check Moodle and OnTrack for detailed information on each of the tasks and what needs to be submitted.

Learning outcomes for this week:

- Create new Inputs using Enhanced Inputs
- Modify the Player character
- Create an Actor Component for an Inventory system
- Create a Spike Trap hazard
- Create Pickup items and power ups



Great job completing your first task last week. We're very happy with the inventory system, and want to make sure we can use it for all of the classes we plan on implementing for this game.

Your first task for this week is to modify the inventory system so that any character can use it. We suggest using an Actor Component for this, and have included some instructions on how to convert it.

Once you've finished that, we need to start setting up some of the items and effects that the player will encounter in the game world. We want some fun powerups for the items the player can pick up in the world, and a hazard for them to avoid.

This document contains the requirements for these next prototypes, and some instructions to help get you started.

Good luck! We're feeling great about this game.

- Project Lead

1. Cloning your Git Repository

We will be building off our project from Week 1, so the first thing we want to do is clone our project so we can do more work on it. If you haven't yet set up your repository for Week 1, please go back and follow those steps.

If you are working on your own device and used it to complete Week 1, you do not need to clone your repository, and can skip this step. You only need to clone your repository if you are working on a new device, such as a lab computer, or a new laptop/computer.

Warning - Git repositories

Even if you are working on a single device for the entire semester, you **must** have a git project correctly set up and regularly updated. This is a required learning outcome, and your work will not be assessed if it is not uploaded to git.

It also ensures that you never lose your work. Unreal will provide no shortage of crashes and strange errors, and you do **not** want to lose your work as a result.

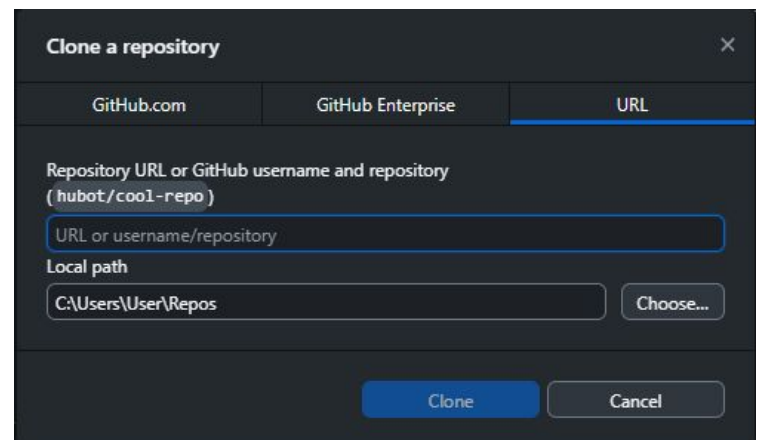
To clone your repository, open GitHub Desktop. You can **skip the login screen** and enter your **name** and **Monash email address** in the next screen.

Next, click **Clone a repository from the Internet** and switch to the URL tab

Now find your repository that you uploaded to [GitLab](#) last week. Click the **Clone** button, and copy the **Clone with HTTPS** link.

Paste this link into Github Desktop in the **Repository URL** textbox, and then set the **Local Path** to the **local documents folder**

(C:\Users\youruser\Documents)



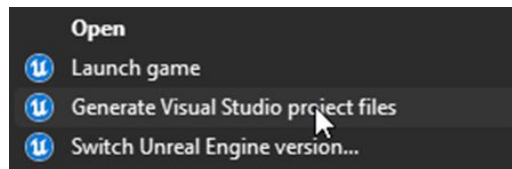
Remember! If you are working on a Lab PC, you need to save your work the **local Documents folder**. Create a new folder in Documents, as we will need an empty folder for your repository.

This will then open a window asking you to log in. **Enter your student email address** and use your **Personal Access Token (PAT)** as the password. Refer back to the [Software Setup Guide](#) for steps on creating a new PAT.

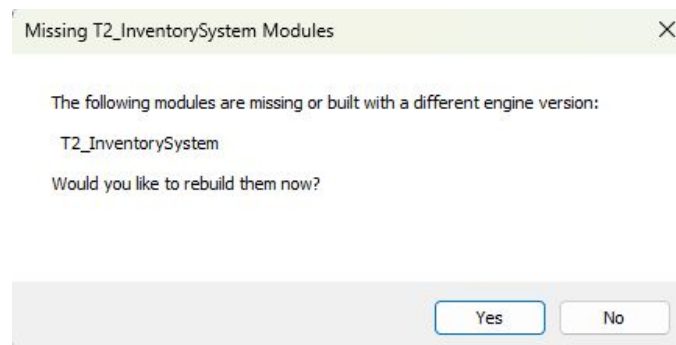
This will clone your repository from the remote to your local device.

2. Open the Unreal Project

Once your project has been successfully cloned, you need to open it up in Unreal. We will follow the same steps as last week. **Right click** on the .uproject file in your project folder and select the context menu option to **Generate Visual Studio Project files**.



Once done, **open** the .uproject file. This will cause a window to Open Saying that parts of the project need to be rebuilt. Select **yes** and wait for Unreal Engine to open.



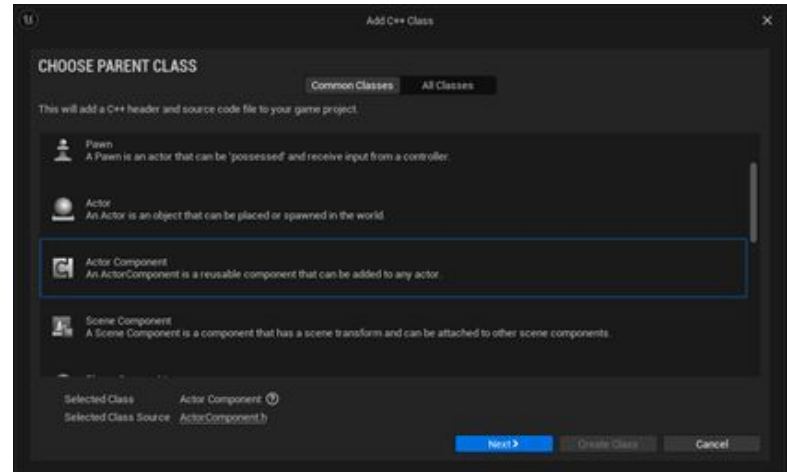
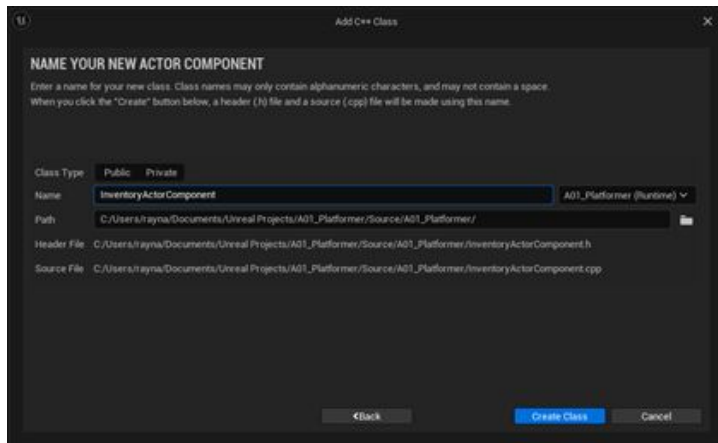
Don't panic if you don't see all your custom classes that you made last week when you first open Unreal. Just hit compile, and they should appear.
Now you should be ready to move on with the tasks for this week.

3. Create an Inventory Actor Component

Last week, we created an inventory for the player using a TArray. Actor Components allow us to modularise functionality, so it can be reused by different classes. We're going to update our Inventory to make it an Actor Component, and make it more flexible. Actor Components allow us to encapsulate all Inventory logic in a single place that can be reused across different characters / NPCs

Start by creating a new class. In the C++ Classes folder in the **Content Browser**, **Right click** and create a **New C++ Class**. Select the parent class **Actor Component** and name it **InventoryActorComponent**.

Wait for the files to generate and open in Rider.



4. Inventory Actor Component Header File

Delete the Tick function

Our Inventory only needs to be updated when we access or change it, not every frame. As such, we can **delete** the **TickComponent** function from both the CPP and Header file.

Now we can declare the variables we need. Our Inventory isn't going to change much from the current implementation, so we only need an array to hold our items, and a variable to represent the maximum number of items that can be stored. Declare these as **protected** variables.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Category="Inventory")
int32 InventorySize;
UPROPERTY()
TArray<UItemObject*> InventoryItems;
```

Add #includes for **ItemObject.h** and **T2_InventorySystemCharacter.h** at the top of the file.

4. Inventory Actor Component Header File

Next, declare the following functions in a **public** section.

```
TArray<UItemObject*> GetAllItems();
UItemObject* GetItemAtIndex(int32 Index);
bool UseItemAtIndex(int32 Index, AT2_InventorySystemCharacter*
Character);
bool DeleteItemAtIndex(int32 Index);
bool AddItem(UItemObject* NewItem);
void CreateDefaultItems();
```

When using, adding or deleting items from our inventory, it's important to know whether or not the function was successful. That's why we return a boolean to let the program know whether or not the task succeeded.

Now you can **generate definitions** for the new functions, and head to the CPP file.

5. Create an Inventory Actor Component (CPP file)

Constructor

Here, we will set our **inventory size** to be **12**. This is the number of slots that fit in our Inventory UI.

While here, we can also set the **bCanEverTick** to be **false**. We don't need to update our inventory every frame.

GetAllItems

For this function, we simply need to return the TArray that contains all of our inventory items.

```
return InventoryItems;
```

GetItemAtIndex

This function will return the item at the provided index, so long as it's a valid item, in which case it will return null.

```
if(Index ≥ InventoryItems.Num() || Index < 0)
{
    return nullptr;
}
return InventoryItems[Index];
```

UseItemAtIndex

We don't have any functionality for our items yet, so for now we can simply **return true**. We'll set this function up later.

5. Create an Inventory Actor Component (CPP file)

DeleteItemAtIndex

We'll attempt to delete the item at the specified index. If we're successful, we'll return true, and if not we'll return false.

```
if(Index ≥ InventoryItems.Num() || Index < 0) {  
    return false;  
}  
InventoryItems.RemoveAt(Index);  
return true;
```

AddItem

For this function, we'll first check if we still have space in the inventory. If we don't, we'll return false, but if we do, we'll add the item and return true.

```
if(InventoryItems.Num() ≥ InventorySize)  
    return false;  
InventoryItems.Add(NewItem);  
return true;
```

CreateDefaultItems

We've already written this code! We can simply **copy** the code from the Character class, and move it here. Remove the final line of code that calls RefreshInventory.

BeginPlay

Finally, we can add a call to our new **CreateDefaultItems** function, to generate some items when we start the game.

```
CreateDefaultItems();
```

6. Update the Character class

Now that we have an inventory component, we need to go back and update our Character class to use it. Go to the **T2_InventorySystemCharacter** files, and make the following changes.

First, delete the **CreateDefaultItems** function from both the Header and CPP file, and remove any references to it.

Next, remove the **InventoryItems TArray** from the Header file. This will cause some errors, which we'll get to in a moment. Ignore them for now.

Before we create a new variable for our InventoryActorComponent, we need to **forward declare** it. We cannot do a normal include for this, as we would create a **circular dependency** between the InventoryActorComponent and the Character class. We do this by adding the "class" keyword before the variable declaration.

```
UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Inventory")
class UInventoryActorComponent* InventoryComponent;
```

We will need the proper include statement in the CPP file.

First, create a **new function** to add an item to the inventory list, that accepts a UItemObject* and returns a boolean.

```
bool AddItem(UItemObject* NewItem);
```

Generate the definition, and we can move over to the **CPP file**.

Before we do anything else, let's add our new include to the top of the file.

```
#include "InventoryActorComponent.h"
```

Ignoring the errors for now, we'll start by creating the Inventory Component inside the **Constructor**. The process is similar to creating new UObjects but uses a different function. This one will require us to also give the component a name that will appear in the component list.

```
InventoryComponent =
CreateDefaultSubobject<UInventoryActorComponent>(TEXT("Inventory
Component"));
```

Now, we'll work through the file fixing up those errors.

GetItemAtIndex

Remove the previous code and replace it with:

```
return InventoryComponent->GetItemAtIndex(Index);
```


6. Update the Character class

DeleteItemAtIndex

Remove the previous code and replace it with:

```
InventoryComponent→DeleteItemAtIndex(Index);  
InventoryWidget→RefreshInventory(InventoryComponent→GetAllItems(  
));
```

AddItem

Implement the AddItem function with the following code:

```
return InventoryComponent→AddItem(NewItem);
```

BeginPlay

Lastly, add the following code to the bottom of BeginPlay, inside the if(APlayerController*...) statement.

```
InventoryWidget→RefreshInventory(InventoryComponent→GetAllItems(  
));
```

At this stage the Inventory Component is fully set up. **Compile** the code and return to Unreal. **Test the game** to ensure there are no errors or issues present. It should function exactly the same as before we started this lab.

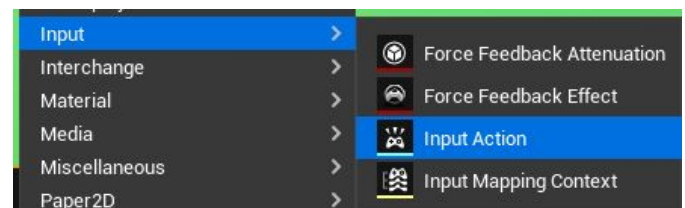
7. Set up user inputs (Enhanced Input)

Unreal uses a system called Enhanced Input for its user interactions. There are two main components, called Input Actions and Input Mapping Context.

Input Actions

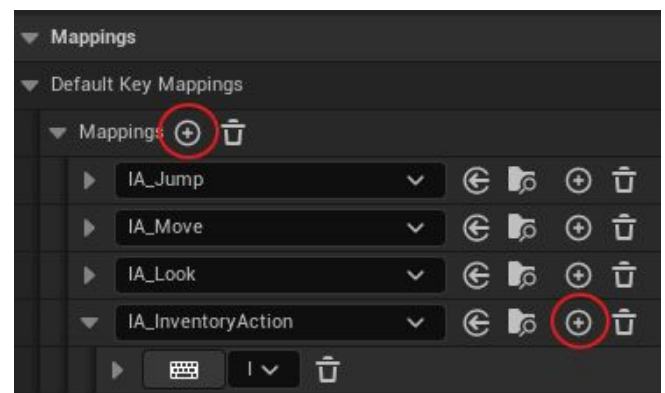
Each action we want the user to be able to perform needs its own file called an Input Action. There are some settings to change modifiers and other behaviour of Inputs Actions, but for simple cases we will create one, attach it to a mapping context, and then bind it in code.

In the Content Drawer, find the **Input** folder and the **Actions** folder inside it. You'll see some existing Actions here. **Right click** anywhere in the folder and select **Input > Input Action** to create a new one. Name it **IA_Inventory**.



Input Mapping Context

An Input Mapping Context (IMC) is a list of controls and actions that we can do. We can create different IMC's for different groups of actions, like if our actions changed while swimming or piloting a vehicle.



Back in the **Input** folder, find the **IMC_Default** file and open it. This has some existing mappings set up for our character movement and look controls. This is typically included in any of the Unreal templates. We just need to add an input mapping for our new action.

Click the **+** icon next to **Mappings** and add our newly created **IA_Inventory** as a new Mapped Action. Next to the new action, click the **+** icon to add a new keybind. For IA_Inventory, select the letter **I** as the key press.

8. Set up user inputs (Header File)

Now that we have created the mapping for our new action, we need to **store a reference** of the action in the character class that will use it, so that the action can be bound to a specific function.

In the **C++ Classes** folder in your Content Drawer, find the **T2_InventorySystemCharacter** class and open it in **Rider**. Add in the **header file** under **public**:

```
UPROPERTY(EditAnywhere)  
UInputAction* InventoryAction;
```

The **UPROPERTY** Macro in Unreal Engine allows us to specify modifiers that the engine can apply to our functions.

EditAnywhere means that the value can be edited from the editor, code, or blueprints.

BlueprintReadWrite allows us to change values from Blueprints.

Category allows us to specify a category it will appear in editor when editing a Blueprint subclass or an instance in the Game World

We'll also need a variable to track the state of our inventory, and a function that will be called when the action is triggered.

```
bool bIsInventoryOpen;
```

```
UFUNCTION()  
void ToggleInventory();
```

In this case, we need the function to use the **UFUNCTION** macro to give the engine special access, as we'll be binding it to a trigger event.

Right click anywhere in the header file and **Generate Code > Definitions**. Now we can head to the CPP File.

9. Set up user inputs (CPP File)

Before we define our new function, we need to make a few changes to existing functions.

Constructor: add a line to set **bIsInventoryOpen** to false.

BeginPlay: delete the line of code that contains **AddToViewport**, and the line that calls **RefreshInventory**.

Next, in the function **SetupPlayerInputComponent** we need to add the bindings of our inputs to specific functions. Add the following inside the statement

if(UEnhancedInputComponent* EnhancedComponent...)

```
// Open/Close Inventory
EnhancedInputComponent→BindAction(InventoryAction,
ETriggerEvent::Started, this,
&AT2_InventorySystemCharacter::ToggleInventory);
```

There are a few types of Trigger Events for buttons, which can be found here:

https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Plugins/EnhancedInput/ETriggerEvent?application_version=5.7

Triggered is activated every single tick if the button is pressed.

Completed is activated when the button is released.

Started is activated the tick that a button is pressed.

Now we can get to the **ToggleInventory** function.

First, we'll want to check if we have a valid PlayerController. If we don't have access to the PlayerController (which typically won't be a problem), we shouldn't continue with the rest of the function, or else the engine may crash.

```
APlayerController* PlayerController =
Cast<APlayerController>(Controller);
```

```
if (!PlayerController)
    return;
```

9. Set up user inputs (CPP File)

Now we need to check if the inventory is currently open or closed using our boolean **bIsInventoryOpen**. If it's showing, we want to remove it, and if it isn't showing, we want to add it to our viewport.

```
if (bIsInventoryOpen) {
    PlayerController→SetInputMode(FInputModeGameOnly());
    InventoryWidget→RemoveFromParent();
    PlayerController→bShowMouseCursor = false;
}
else {
    InventoryWidget→AddToViewport();

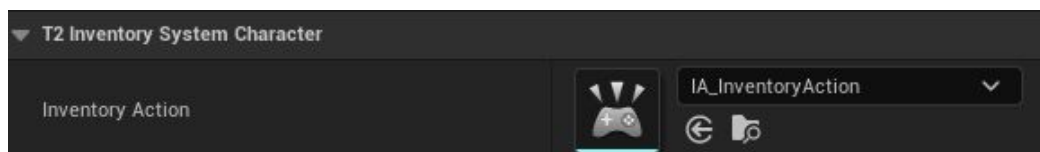
    InventoryWidget→RefreshInventory(InventoryComponent→GetAllItems(
));
    PlayerController→SetInputMode(FInputModeGameAndUI());
    PlayerController→bShowMouseCursor = true;
}
```

And finally, we'll toggle our variable, as we've just changed the state of the inventory.

```
bIsInventoryOpen = !bIsInventoryOpen;
```

We've defined all the functionality we need for now, so we can save the code, return to Unreal, and hit **Compile**.

Open the **BP_ThirdPersonCharacter** file located in **Content Browser > ThirdPerson > Blueprints**. In the details inspector, find the Inventory Action and set the value to be **IA_Inventory**.



Compile the Blueprint and Save. You should now be able to test the game, and open the Inventory by pressing the I key.

10. Create pickup items (Header File)

We now have a functioning inventory system, but we'd like to add a way to pick up these items in the game. Our existing Item class will need to work with a new class so that we can place these items in the game world - for this, we'll be using a brand new Actor class.

Test as you go

As we proceed with our item pickups, we'll work in segmented steps and test each component along the way. This will make it far easier to identify any bugs or errors, as sometimes the Unreal error messages can be a little unhelpful.

Inside your Unreal project, open the **Content Browser** and right click to **Create a New C++ Class**. Select from the list of parent classes the **Actor** class, name the new class **PickupItem** and wait for Rider to open.

First, we'll want to create a variable to represent our model in the game. **Create a StaticMeshComponent**. This is a type of ActorComponent that contains a 3D mesh. We'll want to make sure we can edit this from both the editor and blueprints.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Item
Details")
UStaticMeshComponent* MeshComponent;
```

The Pickup Item will be the object that can be collected in the game world, so it's going to need to have an Item type that can be added to the inventory of the character. Create a variable of type UItem* for this purpose.

Remember to #include ItemObject.h at the top of the file.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Item
Details")
UItemObject* Item;
```

Finally, we'll need a function to detect the collision with the player when they walk through the item. We'll use a UFUNCTION here so that we can use Unreal Events, which will allow us to call this function when the collision occurs.

```
UFUNCTION()
void OnOverlap(UPrimitiveComponent* OverlapComp, AActor*
OtherActor, UPrimitiveComponent* OtherComp,
int32 OtherBodyIndex, bool bFromSweep, const FHitResult&
SweepResult);
```

That's all we need for variables and functions, so **generate definitions** and proceed to the CPP file.

11. Create pickup items (CPP File)

The first step inside of the Constructor is to set `bCanEverTick` to false. As with our Inventory Component, we do not need updates every tick. Once this is done we need to set up the components and attach the mesh to the actor itself.

```
RootComponent = CreateDefaultSubobject<USceneComponent>(TEXT("Root  
Component"));  
MeshComponent =  
CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Mesh  
Component"));  
MeshComponent->SetupAttachment(RootComponent);
```

`SetupAttachment` is used to attach components to others in a nested structure. This is not needed for our Inventory component but it is important for visual elements that can appear in the game world.

BeginPlay

Next we need to tell Unreal to bind our `OnOverlap` function to the event `OnComponentBeginOverlap`. This means when the component collides with another one (i.e. the player) we can call the correct function.

```
MeshComponent->OnComponentBeginOverlap.AddDynamic(this,  
&APickupItem::OnOverlap);
```

We also need to create a default item that can be collected. We will improve this in a later step, so for now we'll just make a default item.

```
if (!Item)  
{  
    Item = NewObject<UItemObject>();  
}
```

11. Create pickup items (CPP file continued)

Next, we'll define that `Overlap` function. First we check to make sure the `OtherActor` exists and is not ourselves, then we attempt to cast to a `Character`. If this succeeds we add the item to the character then destroy the item (as we no longer need it). **Remember to add an include for the character class.**

OnOverlap

```
if (OtherActor && OtherActor != this)
{
    AT2_InventorySystemCharacter* Player =
    Cast<AT2_InventorySystemCharacter>(OtherActor);

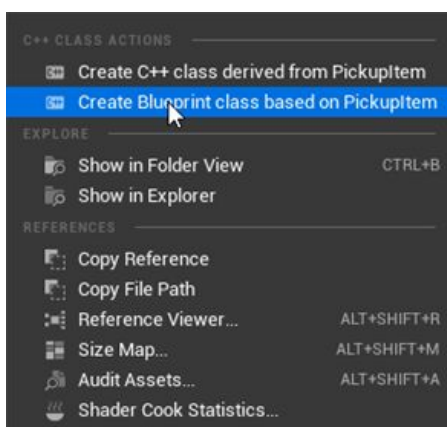
    if (Player)
    {
        if (Player->AddItem(Item))
        {
            Destroy();
        }
    }
}
```

At this point save the code, return to Unreal Engine and **compile**. We now need to create a blueprint subclass of this actor.

12. Create pickup items (Blueprint)

To use this pickup item we will need to create a blueprint subclass. This will allow us to set up the variables we defined in the code.

Open the **C++ Classes** folder, right click on our `PickupItem` C++ class. Select **Create Blueprint class based on PickupItem**, opening a window asking us where to save it. Click on the Blueprints folder and name the new blueprint `BP_PickupItem`. This is how we will name all of our actor blueprints in future.

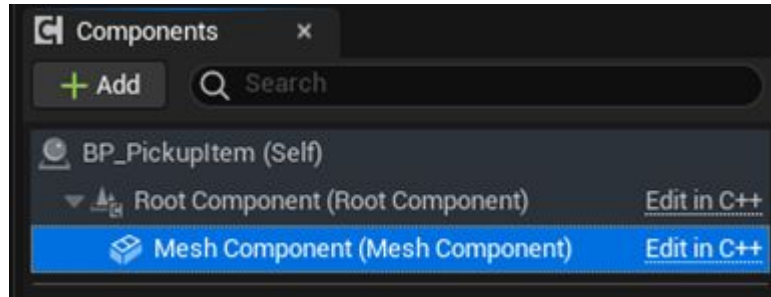


Naming conventions

You may have picked up on the naming conventions we use when creating files in Unreal, such as `BP_PickupItem` and `IA_Inventory`. These prefixes help us keep our project well organised and easy to navigate.

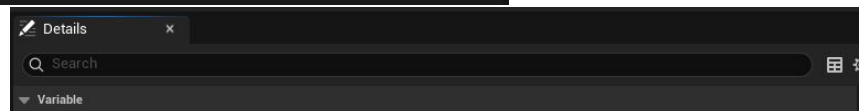
12. Create pickup items (Blueprint continued)

Open the Blueprint Actor and select the **Mesh Component** from the Components list on the left hand side.

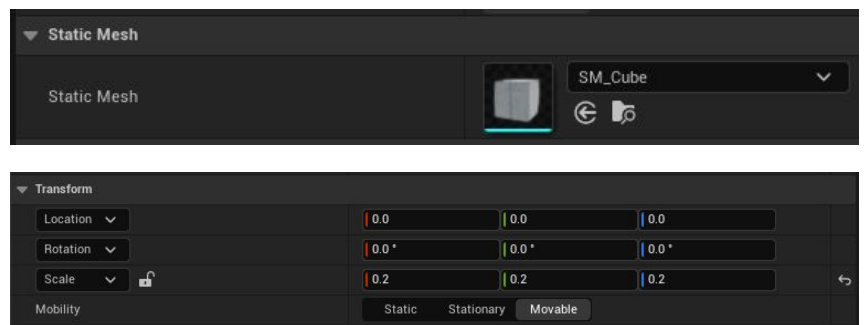


In the Details panel on the right, make the following changes

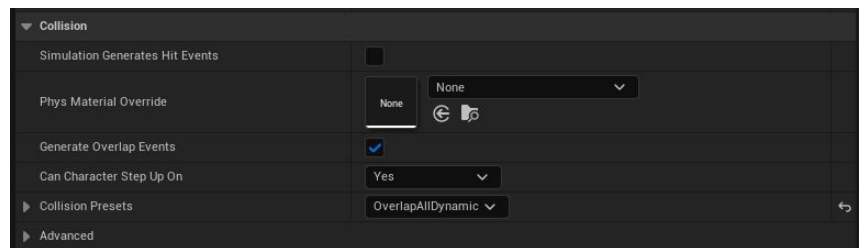
Find Static Mesh and set it to Cube.



Find Transform and set the Scale to 0.2



Find Collision and set Collision Presets to OverlapAllDynamic.



Once done hit **compile** and **save**. Return to the main editor and drag a **BP_ItemPickup** from the Content Browser onto the game world.

Save and hit play. You should be able to run over the object and collect a new item. Check your inventory to ensure it is adding correctly, it'll currently be an empty slot, but if you click on it, it should give you details!

13. Updating the ItemObject class (Header File)

With the PickupItem now working we can turn our attention back to the Item class. Currently item's have a name and description, but no way to use them. They also cannot currently be easily attached to a PickupItem due to how UObjects work by default. We need to make changes to the Item Class. **Open Rider** and then open the **ItemObject** Class header file and make the following changes:

Add the following to the **UCLASS()** Macro at the top of the Item class (the highlighted part).

```
UCLASS(BlueprintType, Blueprintable, EditInlineNew,  
DefaultToInstanced)
```

BlueprintType and Blueprintable enable us to create Blueprint Subclasses of our Items. UObject cannot do this by default unlike Actors, so this is a required step. EditInlineNew and DefaultToInstanced will enable us to quickly and easily set Items from the Editor whenever we place a PickupItem into the world. More information on class macros can be found here: <https://docs.unrealengine.com/5.3/en-US/class-specifiers/>

UCLASS Macros

Whilst we have to put some additional properties into the UCLASS for this class, we do not often need to make these sorts of changes. They only occur when working with UObjects.

Add a forward declaration of the Platformer Character above the class. We need this to be able to use Items correctly.

```
class AT2_InventorySystemCharacter;
```

Create a new virtual function called UseItem to a **public** section. Also **create** an empty **implementation** for the function by **generating a definition** for the function, but leaving it empty.

```
virtual void UseItem(AT2_InventorySystemCharacter* Character);
```

Finally add the following UPROPERTY to both ItemName and ItemDescription so that the values can be changed in editor

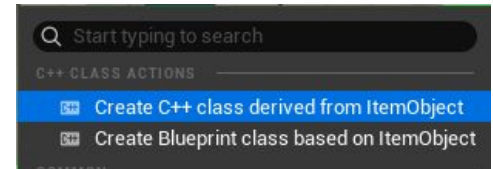
```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
```

With this the changes to the ItemObject Class are done. **Compile** the code and we can begin making the first of two usable Items

14. Creating a SpeedBoost Item (Header File)

The first item we will create is a **Speed Boost**. When the item is used the player will have an increased speed of movement for 10 seconds. We will need to create a C++ subclass of UItem then a Blueprint Subclass of our SpeedBoost.

Begin by right clicking on **ItemObject** in the Content Browser and select **Create C++ class derived from Item**. Name the new class **SpeedBoostItem** and save.



Inside the new header file add the following two variables to a protected section:

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float SpeedBoostAmount;
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float SpeedBoostDuration;
```

Storing amount and duration as variables allows us to create different types of this item. If we wanted to expand the design further we could have lesser, normal and greater versions that each have different values.

To handle the duration of the speed boost we will need to use a **Timer**. Timers allow us to create once off or repeated events that occur at set intervals. For us we will use one to stop the item from working after the duration has ended. More information on timers can be found here:

https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-timers-in-unreal-engine?application_version=5.7

To work with a timer we need to store a handle to one.

```
FTimerHandle BoostTimer;
```

14. Creating a SpeedBoost Item (Header File continued)

The last variable we will need to store is a reference to the Character being affected by our Item. Store this as a pointer as follows:

```
UPROPERTY(EditAnywhere)  
AT2_InventorySystemCharacter* AffectedCharacter;
```

Finally for the header file we need to create a constructor and two functions, UseItem and OnSpeedBoostEnd in a **public** section:

```
USpeedBoostItem();  
virtual void UseItem(AT2_InventorySystemCharacter* Character)  
override;  
UFUNCTION()  
void OnSpeedBoostEnd();
```

15. Creating a SpeedBoost Item (CPP File)

With the Header file finished, **generate definitions** and move to the CPP file.

Constructor

Inside the constructor create some sensible default values for our SpeedBoost. These can be edited later in Blueprint subclasses. In the current implementation the name must match what has been hard-coded into the InventoryWidget.

```
ItemName = "Speed";
ItemDescription = "A mysterious rune. Said to make you faster";
SpeedBoostAmount = 100.f;
SpeedBoostDuration = 1.f;
```

UseItem

Inside the UseItem function we first need to check to ensure that the provided character is a valid target. If it is we apply the SpeedBoost amount and set the Timer

```
if(Character) {
    AffectedCharacter = Character;
    AffectedCharacter->GetCharacterMovement()->MaxWalkSpeed +=
SpeedBoostAmount;
    Character->GetWorld()->GetTimerManager().SetTimer(BoostTimer,
this,
                &USpeedBoostItem::OnSpeedBoostEnd,
                SpeedBoostDuration, false);
}
```

We adjust the speed by changing the value of **MaxWalkSpeed** inside of the Character's Movement Component.

Currently, we don't store the default value of **MaxWalkSpeed**. Once the timer on **SpeedBoost** ends, we won't have a default value to return to. We need another way to end the effect.

To start fixing this, include the **Character class** and the **CharacterMovementComponent.h**

```
#include "T2_InventorySystemCharacter.h"
#include "GameFramework/CharacterMovementComponent.h"
```

15. Creating a SpeedBoost Item (CPP File continued)

OnSpeedBoostEnd

If the character pointer is still valid we subtract the speed back to a default value and set the character pointer to null. This max speed could be changed to a variable to avoid hard-coded values.

```
if(AffectedCharacter) {  
    AffectedCharacter→GetCharacterMovement()→MaxWalkSpeed -=  
    SpeedBoostAmount;  
    AffectedCharacter = nullptr;  
}
```

With this the C++ version of SpeedBoost is done. Save the code, return to Unreal, and hit **Compile**.

16. Creating a SpeedBoost Item (Blueprint File)

With the SpeedBoostItem created, let's create a Blueprint Subclass. We will not be adding any logic but will be using it to set variables in the editor.

Create a new **Blueprint Subclass** of the speed boost item. Name it **BP_SpeedBoostItem** and save it to Blueprints. Open the Blueprint and set its **Speed Boost Amount** to **500** and **Duration** to **10**.

Compile the Blueprint, then save.

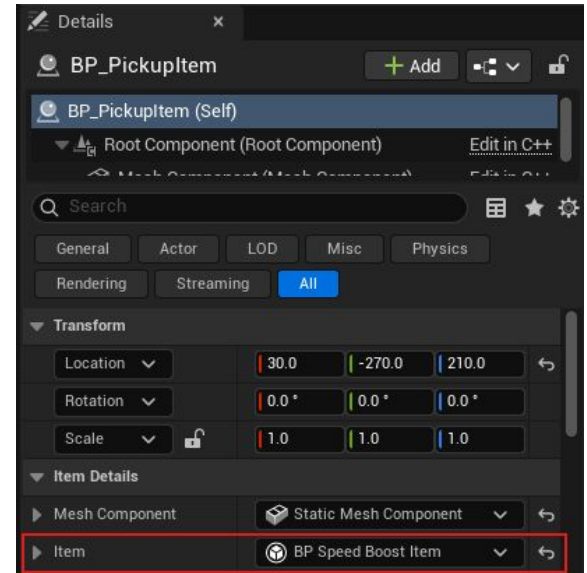
▼ Speed Boost Item	
Speed Boost Amount	500.0
Speed Boost Duration	10.0
Affected Character	None ▼   
▼ Item Object	
Item Name	Speed
Item Description	A mysterious rune. Said to make you faster

17. Creating a SpeedBoost Item (Blueprint File continued)

Next, **select** the **Pickup Item** that has been placed inside the game world. When selected you will be able to change its Details. These changes only change the current instance in the world.

Change its Item to **BP SpeedBoost**, save and hit play. You should now be able to pick up the item and use it... except nothing happens!

We have three final changes we need to make to the Character, Inventory Component and Inventory Widget.



18. Updating the Player to use Items

We need to give the player a function that allows it to use items. Create the following function:

```
void UseItem(int32 Index);
```

Generate an implementation and add the following code to the implementation.

```
InventoryComponent→UseItemAtIndex(Index, this);
InventoryWidget→RefreshInventory(InventoryComponent→GetAllItems(
));
```

19. Updating the Inventory Component

The only change we need to make here is to update the implementation of **UseItemAtIndex** to allow an item to be used. We first check to see if the index is valid and if it is we use the item and remove it from inventory

```
if(Index ≥ InventoryItems.Num() || Index < 0)
{
    return false;
}
UItemObject* UsedItem = InventoryItems[Index];
UsedItem→UseItem(Character);
InventoryItems.RemoveAt(Index);
return true;
```

19. Updating the Inventory Widget

The last change is inside of **InventoryWidget.cpp**. The Use button currently has no functionality attached to it, but it does already have a function bound. Add the following lines of code to the **OnUseButtonClicked** function.

```
Owner→UseItem(SelectedItemIndex);  
SelectedItemIndex = -1;
```

With these changes made save the code, compile and try to test the game again. This time picking up an item and clicking use in Inventory will activate the speed boost! Feel free to increase the speed boost amount a bit to make it more obvious if you're not sure.

21. Make the player healthy

We've got a (currently non-functional) health potion in our inventory, but no health to heal. Before we progress, we're going to quickly give the player a health value that can be used to take and heal damage. We'll set up UI for health in a future lab, but for now we'll just print it as simple text for testing purposes.

To begin open the **T2_InventorySystemCharacter** Header File. Create a new variable for health called **CurrentHealth** and make it a float variable. Give it the **UPROPERTY** **EditAnywhere** and **BlueprintReadWrite**. Set its value to **100**.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)  
float CurrentHealth = 100;
```

Create a new function called **DamagePlayer** that receives a float input called **Damage**. Generate a definition and open the CPP file.

```
void DamagePlayer(float Damage);
```

DamageTank

The **DamageTank** function will be called whenever something needs to apply damage to the player tank. The first thing it needs to do is subtract the damage from the current health.

```
CurrentHealth -= Damage;
```

If the current health hits or drops below zero then the character “dies” in this case we want it to mean that the level is restarted.

```
if(CurrentHealth ≤ 0)  
{  
    APlayerController* PlayerController =  
    Cast<APlayerController>(GetController());  
    PlayerController→RestartLevel();  
}
```

21. Make the player healthy (continued)

Finally, we're going to add a quick way for us to check our character's health. There are a few ways to do this, but for today we'll add an onscreen debug message. This will print the player's updated health each time they take damage.

```
if(GEngine)
{
    GEngine→AddOnScreenDebugMessage(-1, 15.0f, FColor::Yellow,
    FString::Printf(TEXT("Current health: %f"), CurrentHealth));
}
```

For this, we'll get a reference to the engine using the global variable GEngine, and print to the screen. You can read more about this function and the different arguments here: <https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/Engine/UEngine/AddOnScreenDebugMessage>

We've now given the player some health we can use, but no way to hurt them. We'll fix that by making a hazard for them.

22. Create a Spike Trap (Header File)

Now, we're going to make this obstacle course a little more dangerous for the player by making a spike trap. This actor will hurt the player when they stand on it.

Create a new C++ class, name it **SpikeTrapActor** and make sure it is a **subclass of Actor**. For the spike trap, we want to apply damage to the player when they are on top of the trap, which we will detect in much the same way we did with the pickup items. To achieve this we will use a Box Component to see when the player is on top of the spike trap. We will also use a timer to apply damage once every second, instead of once per tick.

We'll start by defining the visual representation of the spike trap (mesh), a box volume that will be slightly larger than the mesh to account for the player standing on the trap, not in it, and a variable for the amount of damage we'll be doing.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)  
UStaticMeshComponent* TrapMesh;
```

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)  
UBoxComponent* DamageVolume;
```

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)  
float DamagePerSecond;
```

We'll also want a **Timer** to handle the rate at which we're damaging the player, and a **reference to the player** that has activated the trap.

```
FTimerHandle DamageTimer;
```

```
UPROPERTY()  
AT2_InventorySystemCharacter* CollidingPlayer;
```

22. Create a Spike Trap (Header File continued)

Last, we'll want some functions to handle the collision event. We'll want one for the **overlap start** as well as one for the **end**, so we know when the player has moved off the trap. We also want a function we'll use to actually **hurt the player** each time the timer runs out.

```
UFUNCTION()  
void OnDamageOverlapBegin(UPrimitiveComponent* OverlappedComp,  
AAActor* OtherActor, UPrimitiveComponent* OtherComp, int32  
OtherBodyIndex,  
bool bFromSweep, const FHitResult& SweepResult);  
  
UFUNCTION()  
void OnDamageOverlapEnd(UPrimitiveComponent* OverlappedComponent,  
AAActor* OtherActor, UPrimitiveComponent* OtherComp, int32  
OtherBodyIndex);  
  
UFUNCTION()  
void DamageTime();
```

Add includes for **Components/UBoxComponent.h** and **T2_InventorySystemCharacter.h** at the top of the header file, then **generate definitions** and move to the CPP file.

23. Create a Spike Trap (CPP File)

Implement the functions in the CPP file as follows.

Constructor

To begin the constructor we need to set default values for **DamagePerSecond** and **CollidingPlayer**. Also be sure to turn **PrimaryActorTick** to **false** (remember, we're using a timer instead).

```
DamagePerSecond = 10;
CollidingPlayer = nullptr;
```

Next the **ActorComponents** need to be set up and attached to the **RootComponent**.

```
RootComponent =
CreateDefaultSubobject<USceneComponent>(TEXT("Root"));
TrapMesh = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Trap
Mesh"));
TrapMesh→SetupAttachment(RootComponent);
DamageVolume = CreateDefaultSubobject<UBoxComponent>(TEXT("Damage
Volume"));
DamageVolume→SetupAttachment(TrapMesh);
```

BeginPlay

We need to bind an event to the **DamageVolume** for both onBegin and onEnd overlaps

```
DamageVolume→OnComponentBeginOverlap.AddDynamic(this,
&ASpikeTrapActor::OnDamageOverlapBegin);
DamageVolume→OnComponentEndOverlap.AddDynamic(this,
&ASpikeTrapActor::OnDamageOverlapEnd);
```

OnDamageBeginOverlap

This function will be called whenever something enters the DamageVolume of this spike trap. Firstly we will attempt to cast the overlapping actor to a Player character..

```
if (OtherActor && OtherActor ≠ this)
{
    AT2_InventorySystemCharacter* TempCharacter =
        Cast<AT2_InventorySystemCharacter>(OtherActor);
}
```

23. Create a Spike Trap (CPP File continued)

If the cast succeeds and the TempCharacter value is not a nullptr, then we can assign the colliding player value and set a timer to go off once every second and call the DamageTime function

```
if(TempCharacter)
{
    CollidingPlayer = TempCharacter;
    GetWorld()→GetTimerManager().SetTimer(DamageTimer, this,
        &ASpikeTrapActor::DamageTime, 1, true);
}
```

OnDamageEndOverlap

This function is called when something stops overlapping with the DamageVolume. In this case we check to see if the CollidingPlayer is the one leaving the overlap. If it is we clear the timer and set CollidingPlayer back to nullptr

```
if(OtherActor == CollidingPlayer)
{
    GetWorld()→GetTimerManager().ClearTimer(DamageTimer);
    CollidingPlayer = nullptr;
}
```

DamageTime

This function is called every time the timer goes off, which in our case is once per second. This function checks to see if we have a colliding player, and if we do then it deals damage to it.

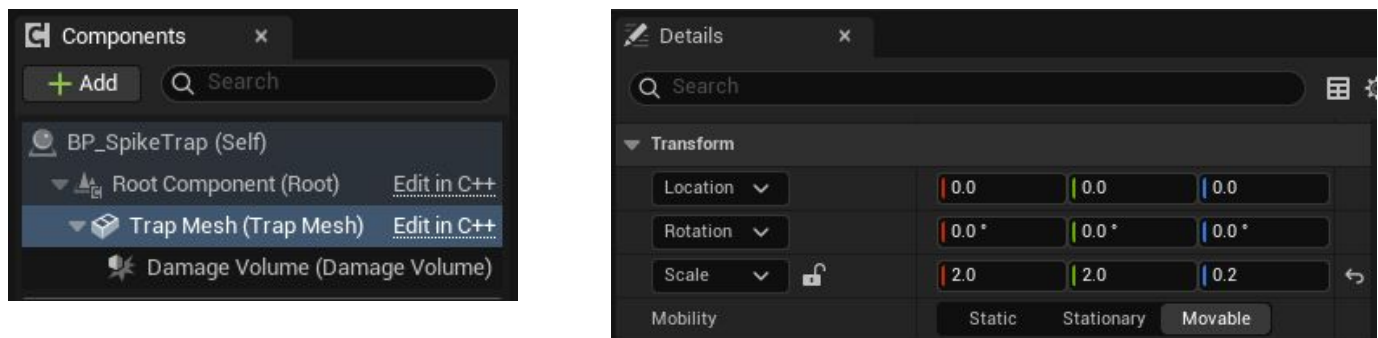
```
if(CollidingPlayer){
    CollidingPlayer→DamagePlayer(DamagePerSecond);
}
```

With this done, our C++ implementation is finished. Save the code, return to Unreal, and hit **Compile**. The final part of this week's lab will involve setting up the spike trap blueprint.

24. Create a Spike Trap (Blueprint)

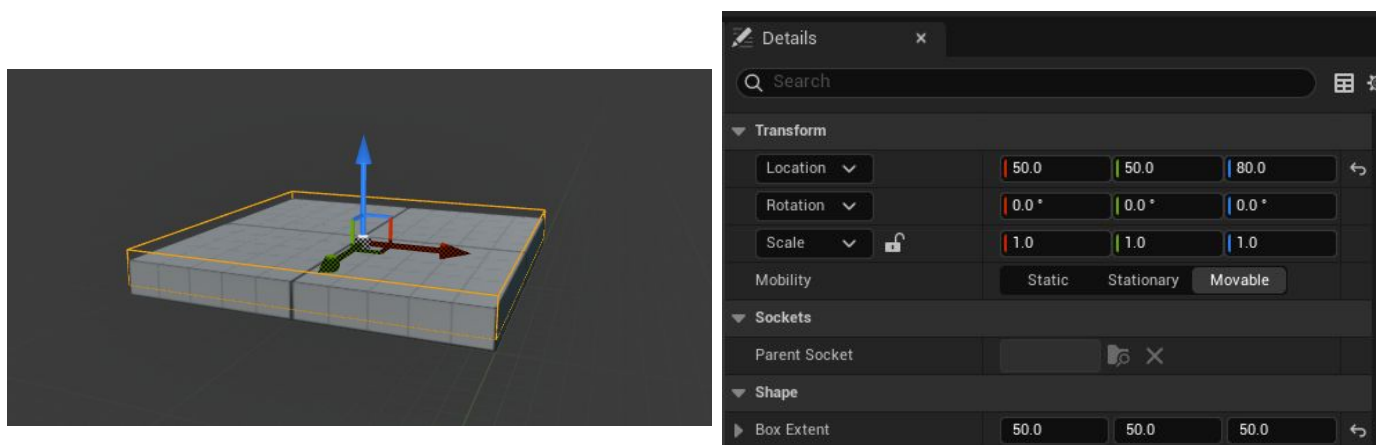
Once again, we'll create a blueprint subclass for use in the game world. **Right click** on the C++ class in Unreal Engine and **create a blueprint subclass**. Remember to use the prefix **BP_** in front of the class name and save it in the **Blueprints folder**.

Open the new subclass and select the **Trap Mesh** from the Components on the left. Then inside the details panel on the right locate **Static Mesh** and change it to **Cube**. To make this more of a flat shape we need to adjust the scale as follows:



Next, select the **Damage Volume** from the Components List. If you select **Viewport** from the top you can see the damage volume as an outline of a box. This box needs to be roughly the same size and shape as platform underneath, with a part sticking out the top.

To achieve this, change the **Location** of the box in the viewport and the **Box Extent** in the details panel until you get the desired result.



These settings may work, but if you selected a different Cube as your Trap Mesh, you may need to change them.

The important part is that the Damage Volume is not entirely hidden inside the Mesh.

Once done **compile, save, and return to the game world**. Place a spike trap into the world. After standing on it for approximately 10 seconds the game should restart if the functionality is working correctly.

26. Git Commit

You're done!

You should now have an improved inventory system, pickup items with working speed boosts, and a spike trap that will kill the player after a few seconds. Run around and test your new features, and then **save** and **close** Unreal.

Be sure to **commit your new changes in Github Desktop** like we did last week. As always, give it a meaningful commit name and description. Try and describe everything you did today in a few dot points so you can easily sort through your code in the future. Once you've committed your work, be sure to **hit the Push button** otherwise the changes won't be sent to the remote repository!

Self Directed Tasks

The following tasks must be completed as part of the pass level tasks for the unit. Please make sure to complete the following and read over [P3 - Working With Actors on OnTrack](#) to ensure that you are completing all required steps

Task 1 - Create a Shrink Powerup

The first self directed task for this week is to create a shrink powerup that allows a player to shrink to 50% size when the powerup is activated. You must make sure that the **powerup effect cannot be stacked multiple times on top of each other**. Make sure you give the ItemName the value "Shrink", as other names will crash the game.

Task 2 - Create a Double Jump Powerup

The second self directed task for this week is to create a double jump powerup. This should grant the ability to double jump for 10 seconds when activated. You must make sure the player can only jump **one extra time per jump**, and cannot repeatedly jump. Be sure to give the power up the ItemName "DoubleJump".

Both tasks will require the following:

- Creating a new Item Type
- Creating a new Use function for the type